

Look Where It Matters: High-Resolution Crops Retrieval for Efficient VLMs

Nimrod Shabtay^{1,2*}, Moshe Kimhi^{3*}, Artem Spector¹, Sivan Haray¹, Ehud Rivlin³,
Chaim Baskin⁴, Raja Giryes², Eli Schwartz¹

¹IBM Research, ²Tel-Aviv University, ³Technion, ⁴Ben-Gurion University

Abstract

Vision-language models (VLMs) typically process images at a native high-resolution, forcing a trade-off between accuracy and computational efficiency: high-resolution inputs capture fine details but incur significant computational costs, while low-resolution inputs offer efficiency at the cost of potentially missing critical visual information, like small text. We present AwaRes, a spatial-on-demand framework that resolves this accuracy–efficiency trade-off by operating on a low-resolution global view and using tool-calling to retrieve only high-resolution segments needed for a given query. We construct supervised data automatically: a judge compares low- vs. high-resolution answers to label whether cropping is needed, and a grounding model localizes the evidence for the correct answer, which we map to a discrete crop set to form multi-turn tool-use trajectories. We train our framework with cold-start SFT followed by multi-turn GRPO with a composite reward that combines semantic answer correctness with explicit crop-cost penalties. Extensive evaluation across six benchmarks demonstrates that AwaRes matches full-resolution accuracy while retaining only 36% of visual tokens.
<https://nimrodshabtay.github.io/AwaRes>

1 Introduction

Vision–language models (VLMs) increasingly rely on high-resolution visual inputs to solve detail-sensitive tasks such as document question answering, chart understanding, and understanding semantics and text in dense natural images. However, high resolution is expensive: the number of visual tokens grows rapidly with image resolution, making high-resolution inference a major bottleneck in practice. Existing approaches to reduce this cost largely fall into several camps. First, *token pruning* methods selectively discard visual tokens to reduce computation (Chen et al., 2024b; Xing et al., 2025; Yang et al., 2025b; Zhang et al., 2025b; Zou et al.,

2025). While effective in principle, they often introduce irregular token patterns and dynamic sequence lengths that can be difficult to translate into end-to-end serving speedups in common inference stacks, such as vLLM (Kwon et al., 2023), where efficiency is tied to predictable sequence length. Second, *resolution escalation* methods (Yang et al., 2025c; Kimhi et al., 2026) learn when to request a higher-resolution view, but typically treat the decision as binary: if more details are needed, the entire high-resolution image is retrieved, wasting computation on regions irrelevant to the question. A key observation is that the demand for high fidelity is usually *spatially sparse*, as can be seen in Fig. 3. Many questions require fine detail in only a small portion of the image: a single value on a chart axis, a specific cell in a table, or a tiny object in the corner of an image. Recent works have begun to exploit this property by training VLMs to selectively zoom into task-relevant regions (Lee et al., 2025; Su et al., 2025; Zheng et al., 2026). However, these methods rely on an iterative reasoning process where latency is dominated by long reasoning trace rather than the visual token budget per round; reducing input resolution alone yields only marginal speedups for these methods, since each additional reasoning step, not pixel count, is the primary cost. We advocate that answering the question of *where* to look matters as much as *whether* to look—and that it can be achieved without heavy-weight reasoning machinery.

We propose AwaRes, a framework that exploits this spatial sparsity via a simple tool-calling interface that targets **high-resolution crop acquisition**. AwaRes processes a low-resolution global view by default, and when additional detail is required, it invokes a tool-call that requests only specific high-resolution sub-regions, and then answers conditioned on both. This multi-turn structure is naturally compatible with KV-caching: computation from the initial low-resolution turn is reused and extended in the crop turn without architectural changes, making AwaRes practical for deployment. We train AwaRes to learn a single *coupled-decision policy (CDP)* that jointly decides (i) *whether* ad-

*Equal contribution.

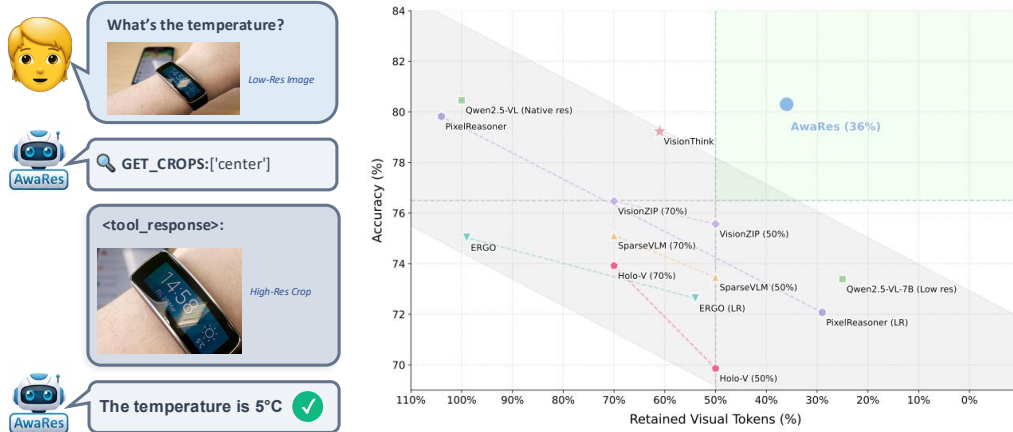


Figure 1: AwaRes overview. Left: Given a low-resolution image, AwaRes uses tool-calling to request only the high-resolution crops needed to answer the query. Right: Accuracy vs. retained visual tokens across six benchmarks. AwaRes performs similarly to native high-resolution (80.3%) while using only 36% of the visual tokens.

ditional resolution is needed and (ii) *where* to acquire it by selecting a subset of crops. Crucially, these decisions are *fused* into the model’s first-turn action: either answer directly, or emit a structured crop request that simultaneously signals escalation and specifies the target regions. For the cold-start phase, we construct the supervision automatically, without manual spatial annotations, by (i) identifying examples where the low-resolution model’s outputs are insufficient using an LLM-as-a-Judge (LaaJ), and (ii) localizing the evidence for the correct answer using a grounding model to produce target crops. We evaluate AwaRes on six benchmarks spanning document understanding and general visual QA. Across these tasks, AwaRes also resolves the same whether-and-where decision much more efficiently, avoiding the reasoning overhead and achieving on average a $5.7\times$ latency reduction over reasoning-based methods, and a $4.4\times$ reduction over the resolution-escalation method (Yang et al., 2025c), while retaining only 36% of visual tokens (RTR) and matching full-resolution accuracy (80.3% vs. 80.46%). Our contributions are listed as follows: (1) We introduce a spatial-on-demand inference framework that acquires targeted high-resolution crops through tool-calling, enabling system-friendly multi-turn KV-cache reuse. (2) We propose an automatic data curation pipeline that produces multi-turn tool-use trajectories without manual annotations. (3) We refine crop usage with multi-turn GRPO using an objective that penalizes unnecessary crop acquisition while discouraging missed crop requests when detail is required.

2 Related Work

Several strategies have emerged to reduce the number of visual tokens in Vision Language Models.

One line of research focuses on dynamic token pruning. Methods such as FastV (Chen et al., 2024b), HoloV (Zou et al., 2025), PyramidDrop (Xing et al., 2025), FitPrune (Ye et al., 2025), TopV (Yang et al., 2025a), SparseVILA (Khaki et al., 2025), IVTP (Huang et al., 2024), LLaVolta (Chen et al., 2024a), and SAINT (Jeddi et al., 2025) discard uninformative tokens within the LLM layers. Alternatively, others prune tokens directly after the vision encoder (Yang et al., 2025b; Vasu et al., 2025; Zhang et al., 2025b; Kimhi et al., 2025b). While effective, these approaches generally commit to a fixed token budget regardless of the specific instance’s complexity. In contrast, our method is fully adaptive, allowing simple images to be processed at minimal cost while allocating more resources only when and where the query demands fine-grained perception.

A second line of work explores resolution selection. CARES (Kimhi et al., 2026) uses an external lightweight model to predict the optimal input resolution, while CROP (Guo et al., 2025) identifies contextual regions of interest via an auxiliary module. Because these methods rely on external components, they require hosting multiple models simultaneously. Our approach, by contrast, natively enables the VLM itself to determine where additional detail is needed.

A third line of work trains VLMs to actively escalate resolution or selectively zoom into task-relevant regions. VisionThink (Yang et al., 2025c) issues a tool call to retrieve the entire high-

resolution image when the low-resolution view is insufficient, wasting computation on irrelevant areas. Alternatively, ZoomEye (Shen et al., 2025), DeepEyes (Zheng et al., 2026), ERGO (Lee et al., 2025), and Pixel Reasoner (Su et al., 2025) achieve spatial selectivity through multiple inference passes or explicit visual operations embedded within long Chain-of-Thought (CoT) reasoning. While effective at improving accuracy, these approaches share a critical bottleneck: their reliance on extended autoregressive reasoning traces or massive full-image token counts severely degrades generation speed. Consequently, they fail to translate into real-world deployment latency benefits. Our method sidesteps this overhead entirely. By emitting a single structured tool call in the first turn to request only targeted high-resolution crops, AwaRes bypasses intermediate reasoning steps and full-image overhead, achieving spatial-on-demand selectivity at a fraction of the inference cost.

3 Method

AwaRes implements *spatial-on-demand* perception via a simple multi-turn interaction: the model first observes a low-resolution global view, and only if needed issues a tool call to retrieve a set of high-resolution crops (Fig. 1). We first formalize this interaction protocol (§3.1), then describe how we automatically curate supervision for the CDP, namely *whether* additional resolution is needed and *where* it matters (Fig. 2; §3.2). Finally, we train in two stages: (i) a *cold-start* supervised fine-tuning (SFT) stage that teaches the tool protocol and yields a supervised *reference policy* π_{ref} (§3.3); and (ii) multi-turn GRPO initialized from π_{ref} and regularized toward it via a KL penalty (§3.4), explicitly optimizing the accuracy–efficiency trade-off.

3.1 Problem setup

Given an image–question–answer triple (I, q, a^*) , the model is first shown a low-resolution view I_{low} (obtained by downsampling I) together with the question q . The model then chooses between two actions:

- (i) **Direct answer:** an answer $\hat{a}$ based on (q, I_{low}) .
- (ii) **Crop request + answer:** Emits a tool call that requests a *subset* of crops from a predefined candidate set, $C_{\text{req}} \subseteq \mathcal{C}$. The tool returns the corresponding high-resolution crop images $\{I_c^{\text{high}}\}_{c \in C_{\text{req}}}$, which are appended to the dialogue context, and the model produces the final answer $\hat{a}$ conditioned

on the full multi-turn history.

A fused coupled-decision policy: We parameterize a single policy over high resolution request, and localized crop selection:

$$\pi_{\theta}(C \mid q, I_{\text{low}}), \quad C \subseteq \mathcal{C}, \quad (1)$$

where $C = \emptyset$ corresponds to *no tool call* (answer directly) and $C \neq \emptyset$ corresponds to *escalation with localization*. Under this view, “when to crop” is the marginal event $\mathbb{1}[C \neq \emptyset]$, while “where to crop” is the conditional distribution over C given $C \neq \emptyset$. The two are inherently coupled: the value of escalating depends on *which* regions will be retrieved, since inaccurate localization can waste compute without improving answer correctness. This interface targets efficiency by restricting high-resolution perception to a small number of structured regions, while preserving the low-resolution global context throughout the interaction (See Fig. 1 for a conversation example).

3.2 Automatic supervision for crop requests

A key challenge is to supervise two coupled decisions: whether the low-resolution view is insufficient, and **where** to crop when additional detail is needed. We generate this supervision to initiate the model to learn a reference policy $\pi_{\theta_{\text{ref}}}$ in an automatic fashion using the three-stage pipeline (Illustrated in Fig. 2):

Stage 1: resolution-sufficiency labeling (when to crop). For each example (I, q, a^*) we utilize a base VLM T on both the low-resolution and full-resolution inputs:

$$\hat{a}_{\text{low}} = T(q, I_{\text{low}}), \quad \hat{a}_{\text{full}} = T(q, I). \quad (2)$$

Because $\hat{a}_{\text{low}}$ and $\hat{a}_{\text{full}}$ may differ in form though semantically correct, direct string matching (exact match) with a^* is unreliable. Instead, we use an LaaJ (LLaMA-3.3-70B (Grattafiori et al., 2024)) to compare both predictions to the ground truth a^* . If it judges $\hat{a}_{\text{low}}$ as correct (or ties it with $\hat{a}_{\text{full}}$), we label the example as no crop needed LR; otherwise we label it as HR.

Stage 2: crop target construction (where to crop). For examples labeled HR, we identify the region that contains the visual evidence needed to answer (q, a^*) . We prompt a grounding model G (namely, Qwen3-VL-A235B-A22B (Bai et al., 2025a)) to localize the evidence and return a bounding box $b = (x_1, y_1, x_2, y_2)$ in the coordinate system of the original image.

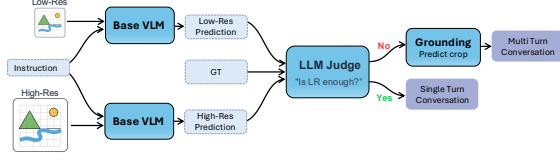


Figure 2: **Overview of the automatic supervision pipeline.** Each sample is processed at two resolutions; an LLM judge determines resolution sufficiency by comparing predictions to ground truth. Sufficient cases yield single-turn conversations, while insufficient cases are routed to a grounding model for crop localization, producing multi-turn trajectories with tool-calling.

We then map b to our discrete crop candidate set $\mathcal{C}$, which includes four quadrants, a center crop, four merged half-image regions (top/bottom/left/right), and a full-image. We define the target crop subset as

$$\mathcal{C}^* = \{c \in \mathcal{C} \mid \text{IoU}(b, c) \geq \tau\}, \quad (3)$$

where $\tau = 0.5$ is the IoU threshold. Fig. 3 shows a representative example, and Fig. 5 (left side) summarizes the empirical distribution of selected crops in the curated training set.

Stage 3: supervised tool-use trajectories. The procedure above yields two types of training transcripts:

Direct-answer trajectories (LR). The model observes (q, I_{low}) and is supervised to output a^* in a single turn.

Tool-call-then-answer trajectories (HR). In the first turn, the model issue a tool call selecting $\mathcal{C}^*$. After the tool returns $\{I_c^{\text{high}}\}_{c \in \mathcal{C}^*}$, the model is trained to produces a^* in a second turn conditioned on both the low-resolution and the retrieved high-resolution crops.

This curation pipeline produces multi-turn tool-use supervision at scale in order to learn an initial reference policy $\pi_{\theta_{\text{ref}}}$, while keeping the crop interface structured and deployment-friendly (Fig. 2). We provide additional details in the Appendix.

3.3 Cold-start (SFT)

We cold-start our crop-request policy by supervised fine-tuning (SFT) on the mixture of direct-answer and tool-call-then-answer trajectories produced in §3.2.

This stage serves two purposes: (i) teach the model to follow the multi-turn tool-calling protocol and learn the coupled decisions (*whether* additional detail is needed and *where* it matters), and (ii) produce a strong supervised *reference policy* π_{ref} that we later use for KL-regularized GRPO (§3.4).

Let $y_{1:T}$ denote the assistant tokens in a supervised transcript, and let h_t be the dialogue history at step t (including (q, I_{low}) , any previously generated tokens, and tool outputs if a crop request occurred). We minimize a weighted negative log-likelihood:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t=1}^T w_t \log \pi_{\theta}(y_t \mid h_t) \quad (4)$$

The tool-call turn, despite having small numbers of tokens, fully specifies the CDP action and carry disproportionate control over both efficiency and downstream answer quality. Upweighting this turn therefore directly stabilizes learning of the *fused* first-turn decision. After SFT, we freeze the resulting model as the reference policy π_{ref} and initialize GRPO from it.

3.4 Multi-turn GRPO

After the cold-start SFT stage, the model reliably follows the tool protocol but tends to *over-request* crops even when I_{low} is sufficient. We therefore apply Group Relative Policy Optimization (GRPO) on full multi-turn interactions to explicitly optimize the accuracy–efficiency trade-off.

We denote by π_{ref} the *frozen* SFT policy $\pi_{\theta_{\text{ref}}}$ obtained from the SFT. GRPO is initialized from π_{ref} and uses a KL penalty to keep π_{θ} close to π_{ref} while improving tool usage.

Rollouts and trajectories. Given an input prompt $x = (q, I_{\text{low}})$, the policy π_{θ} generates first turn that may include a crop tool call. The requested crops are appended to the dialogue context, and generation continues until a final answer $\hat{a}$ is produced. We treat only assistant tokens as actions; tool outputs are treated as observations. Thus, each rollout yields a multi-turn trajectory τ consisting of assistant actions interleaved with tool observations, ending with $\hat{a}$.

Unlike supervised training with dense per-token loss, GRPO enables optimization with task-specific rewards that directly target improved tool usage.

3.4.1 Reward design.

We assign a single scalar reward to the completed trajectory τ , composed from two components:

$$R(\tau) = w_a \cdot R_{\text{ans}}(\hat{a}, a^*) - C_{\text{tool}}(C, y), \quad (5)$$

Answer reward ($R_{\text{ans}}(\hat{a}, a^*)$): measures semantic correctness using the cosine similarity between

Prompt:

What website is being advertised on the plane?

Ground Truth Answer:

Jetstar.com



Prompt:

What is the Grand Total for Net Block As of 31.3.2012?

Ground Truth Answer:

52708.86

Figure 3: **Crop annotation example.** Left: low-resolution input where text is illegible. Middle: grounding model G predicted bounding box localizing the answer region. Right: selected high-resolution crop enabling correct response (best viewed when zoomed in).

sentence-transformer embeddings of $\hat{a}$ and a^* , where w_a scales the answer reward relative to the tool cost.

Tool-use cost: Penalize tool usage with an asymmetric cost:

$$C_{\text{tool}}(C, y) = \begin{cases} \alpha_{\text{miss}} & \text{if } y = \text{HR and } C = \emptyset \quad (\text{missed tool-call}) \\ \alpha_{\text{use}} + \lambda \|C\| & \text{if } C \neq \emptyset \quad (\text{tool usage}) \\ 0 & \text{if } y = \text{LR and } C = \emptyset, \end{cases} \quad (6)$$

This asymmetry biases the policy toward *recall* in tool invocation: missing a necessary crop request is penalized more heavily than making an unnecessary request. When the tool is used, we additionally penalize the *amount of high-resolution evidence requested* via $\|C\|$, defined as the total fraction of image area covered by the selected crops. This encourages the policy to prefer smaller crops when they suffice. Importantly, the cost depends on *how much* is requested but remains agnostic to *which* specific region is chosen, allowing the GRPO to explore alternative policies.

3.4.2 GRPO optimization.

For each prompt x , we sample a group of G trajectories $\{\tau_1, \dots, \tau_G\}$ from the current policy π_θ . Each trajectory τ_i consists of a sequence of assistant tokens (actions) interleaved with tool observations, culminating in a final answer. We compute the advantage for each trajectory using the group-relative baseline:

$$\hat{A}_i = \frac{R(\tau_i) - \mu_G}{\sigma_G + \epsilon}, \quad (7)$$

where $R(\tau_i)$ is the total reward for trajectory τ_i , and μ_G, σ_G are the mean and standard deviation of rewards within the group.

We optimize a PPO-style clipped objective with KL regularization to the reference policy:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|\tau_i|} \sum_{t=1}^{|\tau_i|} \min(r_t^{(i)} \hat{A}_i, \text{clip}(r_t^{(i)}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \right] \quad (8)$$

where $r_t^{(i)} = \frac{\pi_\theta(a_t^{(i)} | x, a_{<t}^{(i)})}{\pi_{\text{old}}(a_t^{(i)} | x, a_{<t}^{(i)})}$ is the importance sampling ratio, ϵ is the clipping threshold, and β con-

trols the strength of the KL divergence penalty against a reference policy π_{ref} .

As in PPO-style updates, π_{old} denotes a snapshot of the policy before the current GRPO update step. The KL term is computed over assistant-token distributions along the sampled trajectories, encouraging stable improvements over π_{ref} .

3.5 Inference

At test time, we follow the same interaction protocol used during training. The model receives (q, I_{low}) and either answers directly or emits a tool call selecting a crop subset $C \subseteq \mathcal{C}$. If a tool call occurs, the corresponding high-resolution crops are appended to the dialogue context while retaining I_{low} , and the model produces the final answer in a second turn.

This results in two possible inference paths: a single prefill pass for queries answerable from the low-resolution view, or two prefill passes when high-resolution detail is required. In the latter case, the model benefits from both the global context preserved in I_{low} and the fine-grained detail in the requested crops. When a second turn is required the low-resolution view and the query are already in the KV cache saving on the required compute. Crucially, the decision of which path to take, and which regions to acquire - is made entirely by the learned policy, requiring no external heuristics or task-specific thresholds.

4 Experimental Results

We evaluate AwaRes on six benchmarks spanning document understanding and general visual QA, and compare against both fixed-budget token-pruning methods and adaptive resolution-escalation baselines. We report (i) the dataset metric from `lmms-eval` (Zhang et al., 2025a) and (ii) an *Retain Token Ratio* (RTR), defined as the fraction of visual tokens processed relative to the full-resolution baseline. RTR directly reflects the model’s first-turn *coupled-decision policy* (answer directly vs. request crops), while accuracy reflects the quality of the full multi-turn interaction. We first describe our evaluation protocol 4.1, evaluated datasets 4.2 and implementation details 4.3. Then, we provide a detailed discussion of the main results 4.5 and conclude by extensive ablations 4.6.

4.1 Evaluation Protocol

All models are evaluated using `lmms-eval` (Zhang et al., 2025a), and we report the per-dataset metrics

provided by the framework.

Retain Token Ratio (RTR): We measure efficiency via *visual token* usage, which dominates compute and KV-cache memory at high resolution. Let T_i be the visual tokens a sample i consumes across *all* turns and T_{full} those of one full-resolution pass with the baseline model; then $\text{RTR}_i = T_i/T_{\text{full}}$ and $\text{RTR} = \mathbb{E}_i[\text{RTR}_i]$. Fixed-budget methods are configured to retain 50% or 70% of the full-resolution tokens; for adaptive methods we count the tokens actually consumed per sample and average over the dataset.

Latency: When reporting wall-clock time (Fig. 4), we measure end-to-end per-sample latency, including both turns for methods that invoke the crop tool.

4.2 Datasets

We curated a diverse training set comprising 10K samples from each of five publicly available training sets: ChartQA (Masry et al., 2022), DocVQA (Mathew et al., 2021), TextVQA (Singh et al., 2019), LLaVA-Multi (Jiang et al., 2024), and VisionThink-Smart (Yang et al., 2025c). We also collected 2k samples with the same distribution as a validation set. The SFT phase using a subset of the training set (5k samples from each dataset), while the GRPO uses all collected examples. This mixture spans both document understanding and natural image domains.

We evaluate on six benchmarks spanning natural image understanding-RealWorldQA (xAI, 2024) (real-world spatial reasoning), POPE (Li et al., 2023) (object hallucination), and V^* -Bench (Wu and Xie, 2024) (visual search in high-resolution scenes)-and document understanding-ChartQA (Masry et al., 2022) (reasoning over charts), DocVQA (Mathew et al., 2021) (forms, tables, letters, and handwriting), and OCRBench (Liu et al., 2024) (text recognition and text-centric reasoning). full details are in the appendix.

4.3 Implementation Details

We conduct experiments based on Qwen2.5-VL-7B-Instruct (Bai et al., 2025b); all compared methods are built on the same base VLM to isolate the impact of efficiency mechanisms.

Unless otherwise specified, each sample is first processed at a low-resolution setting I_{low} has height and width divided by 2, corresponding to the “LR” baseline in Table 1 (RTR=0.25). When a crop is requested, the crop image(s) are rendered at the

native high-resolution token density of the base model.

We use the discrete crop set $\mathcal{C}$ from §3.2 (four quadrants, four half-image regions, a center crop, and the full image). Requested crops are appended to the context together with I_{low} .

For training, we use HuggingFace TRL (von Werra et al., 2020) for SFT and GRPO, adapting TRL’s GRPO trainer to support multi-image and multi-turn conversations. For the SFT phase we use a total batch size of 16 with learning rate 1×10^{-4} and LoRA rank 8. $w_t = 5$ only for the tool-call turn and 1 otherwise. For the GRPO phase we use a total batch size of 64 with $G=8$ generations per sample, learning rate 1×10^{-5} , and LoRA rank 8. $w_a = 10$, $\alpha_{\text{miss}} = 2$, $\alpha_{\text{use}} = 0.25$ and $\lambda = 0.01$.

4.4 Baselines

All baselines are based on Qwen-2.5-VL-7B (Bai et al., 2025b). **Fixed-budget token pruning/compression:** VisionZip (Yang et al., 2025b), SparseVLM (Zhang et al., 2025b), and HoloV (Zou et al., 2025), training-free inference-time methods configured to retain 50% or 70% of the full-resolution visual tokens. **Dynamic methods:** VisionThink (Yang et al., 2025c), which escalates a low-resolution pass to the *full* high-resolution image, and the reasoning-based ERGO, Deep-Eyes, and Pixel-Reasoner (Lee et al., 2025; Zheng et al., 2026; Su et al., 2025), which can request a crop from it. AwaRes instead escalates *spatially*, via a short tool call.

4.5 Main Results

Table 1 reports task performance and efficiency. AwaRes achieves an average score of 80.30, matching the full-resolution baseline (80.46) while using only $0.36\times$ the visual tokens on average. This indicates that the learned *coupled-decision policy* answers directly from the low-resolution overview when possible, and invokes targeted crops only when necessary.

Compared to fixed-budget pruning methods, AwaRes consistently attains higher accuracy at comparable or lower RTR. For example, at 70% retained tokens, VisionZip trails AwaRes by over 4 percent on average (76.47 vs. 80.30), reflecting the limitation of a global, sample-agnostic token budget.

Against the adaptive escalation baseline VisionThink (Yang et al., 2025c), AwaRes improves both accuracy (80.30 vs. 79.23) and efficiency (RTR

0.36 vs. 0.61). Notably, on ChartQA AwaRes slightly exceeds the full-resolution baseline (+0.84 percent) while reducing RTR to 0.32, whereas VisionThink often performs two visual passes and exceeds baseline compute (RTR=1.15).

4.5.1 Beyond Token Efficiency

Although RTR captures the dominant visual compute, end-to-end latency also depends on the number of autoregressive text tokens generated: other dynamic methods decide whether to escalate through extended reasoning traces, which inflates decoding cost even when the final answer is short, whereas AwaRes encodes the decision in a short, structured tool call and reuses the KV cache between turns.

Fig. 4 reports average wall-clock time for all dynamic methods, with a reference line at one second—a practical requirement for interactive applications. AwaRes is the only method consistently below it, averaging 0.61s across the six benchmarks, and is $2\text{--}12\times$ faster than every baseline. The figure also reports each baseline’s self-speedup from a low-resolution first turn (“LR”): ERGO benefits most ($1.7\times$, $2.04\text{ s} \rightarrow 1.20\text{ s}$), still nearly twice AwaRes’s latency, while PixelReasoner and DeepEyes gain only $1.1\times$ and $1.2\times$, indicating their latency is dominated by text decoding and multi-turn overhead rather than visual encoding. Two choices drive this: the resolution decision costs a single decoding step rather than a reasoning trace, and the loop terminates early when the low-resolution view already suffices, avoiding a second forward pass. Under a production-grade engine (vLLM), AwaRes averages 0.24s per sample, 31% faster than the high-resolution baseline, so the token savings do translate into deployment latency.

4.5.2 From Over-Calling to Looking Where It Matters

Fig. 5 illustrates how the model’s *first-turn crop-request policy* evolves from grounded supervision (denoted as Oracle GT) through SFT to GRPO. The SFT designed to introduce the tool-calling protocol, and as so, the policy becomes conservative with respect to accuracy and therefore *over-invokes* the crop tool: the probability of taking the no-call action (LR) drops sharply ($80.8\% \rightarrow 46.3\%$), while escalation to the full-image crop (“All”) is substantially inflated ($3.4\% \rightarrow 16.6\%$). This behavior is consistent with imitation-style training that prioritizes adhering to the tool protocol, even in cases

Table 1: **Main results across vision-language benchmarks.** We compare AwaRes against fixed-ratio token-pruning methods (VisionZIP, SparseVLM, Holo-V) and dynamic, crop-inspired methods (ERGO, DeepEyes, PixelReasoner, VisionThink). Retain Token Ratios (RTR) denote the fraction of visual tokens retained. “LR” indicates the first turn uses a low-resolution image. AwaRes matches the full-resolution baseline (Qwen2.5-VL-7B) while using only 36% of the visual tokens, outperforming all efficient alternatives. Best results per column in **bold**.

Model	ChartQA		DocVQA		OCRBench		POPE		RealWorld		V* Bench		Average	
	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$
<i>Base models</i>														
Qwen2.5-VL-7B	79.80	1.00	94.00	1.00	81.10	1.00	87.87	1.00	68.80	1.00	71.20	1.00	80.46	1.00
Qwen2.5-VL-7B-LR	65.00	0.25	91.00	0.25	70.70	0.25	84.41	0.25	66.00	0.25	63.20	0.25	73.39	0.25
<i>Token-pruning methods (fixed ratio)</i>														
Holo-V (50%)	62.04	0.50	70.77	0.50	68.20	0.50	86.54	0.50	67.19	0.50	64.40	0.50	69.86	0.50
Holo-V (70%)	69.32	0.70	76.40	0.70	72.40	0.70	87.37	0.70	68.89	0.70	69.11	0.70	73.92	0.70
SparseVLM (50%)	73.20	0.50	83.60	0.50	75.60	0.50	85.50	0.50	68.40	0.50	54.45	0.50	73.46	0.50
SparseVLM (70%)	75.80	0.70	87.20	0.70	79.30	0.70	85.40	0.70	68.50	0.70	54.45	0.70	75.11	0.70
VisionZIP (50%)	74.76	0.50	89.39	0.50	69.20	0.50	87.56	0.50	66.01	0.50	66.49	0.50	75.57	0.50
VisionZIP (70%)	76.72	0.70	90.75	0.70	72.70	0.70	87.86	0.70	64.84	0.70	65.97	0.70	76.47	0.70
<i>Dynamic / crop-inspired methods</i>														
ERGO	80.30	1.293	59.80	0.435	74.60	1.597	86.20	1.277	65.50	0.933	83.80	0.401	75.03	0.99
ERGO (LR)	73.40	0.614	60.40	0.380	72.10	0.840	84.40	0.577	64.30	0.466	81.20	0.366	72.63	0.54
DeepEyes	61.76	1.090	82.95	1.012	80.50	1.350	34.40	1.260	65.75	1.090	80.10	1.001	67.58	1.13
DeepEyes (LR)	51.04	0.277	82.04	0.253	66.80	0.529	32.74	0.296	61.05	0.277	72.25	0.253	60.99	0.31
PixelReasoner	87.90	1.010	92.93	1.020	70.80	1.020	88.30	1.130	54.20	1.000	84.81	1.050	79.82	1.04
PixelReasoner (LR)	74.00	0.270	92.15	0.270	59.30	0.300	85.06	0.400	51.76	0.250	70.15	0.270	72.07	0.29
VisionThink	79.90	1.150	90.35	0.320	80.10	0.830	86.70	0.340	66.60	0.550	71.73	0.490	79.23	0.61
AwaRes	80.64	0.320	94.43	0.280	81.30	0.420	85.73	0.270	68.50	0.430	71.20	0.420	80.30	0.36

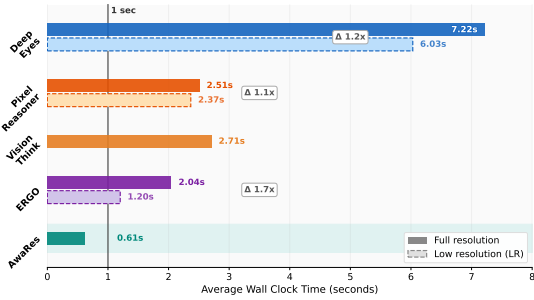
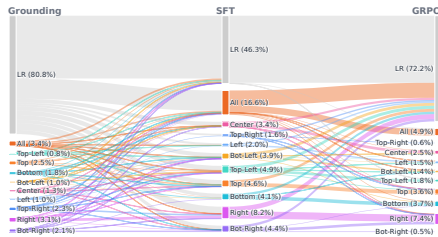


Figure 4: **Latency breakdown.** Full vs. low-resolution (LR) first-turn input; solid bars are full-resolution wall-clock time, dashed bars the LR variant. The small speedups ($\Delta 1.1$ – $1.7\times$) show that input token count is not the main bottleneck: cost is dominated by extended reasoning or iterative cropping. AwaRes reaches 0.61s via a single short tool call, avoiding verbose generation.



4.6.2 Tool introduction

Table 6 studies how SFT recipes affect the emergence of reliable crop-tool usage. The default optimizes each assistant message independently, whereas **Traj** performs trajectory-level SFT over the entire two-turn interaction as a single sample, and **Phased** trains on the tool-call turns first. Trajectory-level training improves both accuracy and efficiency over split training (77.90 vs. 75.15 and 0.43 vs. 0.36), and raising the tool-turn weight to $w_t=5$ improves accuracy further (79.70) at a higher RTR (0.49), suggesting that emphasizing the first-turn decision encourages more confident tool invocation. **Upsampled** variants reduce RTR (to 0.33) but also accuracy, reflecting the expected trade-off. We therefore initialize GRPO from trajectory-level SFT with $w_t=5$.

4.6.3 Tool Optimization rewards

We ablate the GRPO reward components, which combine a sentence-transformer answer-similarity reward with a tool-use cost (Table 9). Removing the crop-area cost raises RTR from 0.36 to 0.42 ($\lambda = 0$), and removing the crop cost entirely raises it to 0.51; the increase is bounded by the SFT initialization and the KL term, which keep the policy close to a reference that already uses the tool sensibly. ANLS and LLM-as-a-judge correctness rewards perform similarly, since we optimize short answers rather than lengthy reasoning traces; sentence-transformer similarity balances the semantic sensitivity ANLS lacks against the robustness an LLM judge lacks.

4.6.4 The effect of cold start initialization

An alternative to our two-stage approach is to run RL directly from the base model and let it learn tool use from scratch during GRPO. Trained this way for 3 epochs on the same data (GRPO only, Tab. 9), the model barely uses the tool, falls back to low-resolution processing, and loses accuracy. SFT alone reaches good accuracy but high RTR from over-calling, and it is a double-edged sword: on an external validation set (Fig. 5) the SFT model has overfitted to the training data and shifted its policy substantially. GRPO is therefore not only exploratory but also recovers from that overfitting.

5 Conclusion

We presented AwaRes, a spatial-on-demand inference framework for VLMs that keeps a low-resolution global view and retrieves high-resolution

crops only when the query requires them, through a deployment-friendly tool-calling interface with multi-turn KV-cache reuse. Training needs no manual spatial supervision: an automatic curation pipeline supplies the data, and we train with cold-start SFT followed by multi-turn GRPO under a reward combining answer correctness and crop-usage penalties. Across six benchmarks, AwaRes matches full-resolution accuracy with far fewer visual tokens and lower latency than both dynamic and fixed-budget baselines.

Limitations

Our framework acquires high-resolution evidence from a *fixed, discrete* crop set (four quadrants, four half-image regions, a center crop, and the full image). This keeps the tool interface structured and deployment-friendly, but it also caps spatial precision: when the answer-bearing evidence straddles crop boundaries or occupies a region much smaller than a quadrant, the policy must request more area than strictly necessary. Extending the interface to continuous bounding boxes is left to future work.

The resolution-sufficiency label comes from an LLM judge and the crop target from a grounding model; both make mistakes (Appendix A shows representative failures), and the resulting label noise propagates into the cold-start policy. We measure inter-model agreement rather than validating against human annotations.

We evaluate a single VLM (Qwen2.5-VL-7B) and latency numbers are measured on a single NVIDIA H100-80GB GPU under HuggingFace and vLLM, meaning absolute wall-clock times may differ on other serving stacks.

References

- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. 2025a. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*.
- Shuai Bai, Ke qin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, and 8 others. 2025b. [Qwen2.5-vl technical report](#). *ArXiv*, abs/2502.13923.
- Jieneng Chen, Luoxin Ye, Ju He, Zhao-Yang Wang, Daniel Khashabi, and Alan Yuille. 2024a. Efficient large multi-modal models via visual context compression. In *NeurIPS*.
- Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. 2024b. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision*.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Jiawei Guo, Feifei Zhai, Pu Jian, Qianrun Wei, and Yu Zhou. 2025. [Crop: Contextual region-oriented visual token pruning](#). *Preprint*, arXiv:2505.21233.
- Kai Huang, Hao Zou, Ye Xi, BoChen Wang, Zhen Xie, and Liang Yu. 2024. Ivtp: Instruction-guided visual token pruning for large vision-language models. In *ECCV*, pages 214–230.
- Ahmadreza Jeddi, Negin Baghbanzadeh, Elham Dolatabadi, and Babak Taati. 2025. [Similarity-aware token pruning: Your vlm but faster](#). *Preprint*, arXiv:2503.11549.
- Dongfu Jiang, Xuan He, Huaye Zeng, Cong Wei, Max Ku, Qian Liu, and Wenhui Chen. 2024. Mantis: Interleaved multi-image instruction tuning. *arXiv preprint arXiv:2405.01483*.
- Samir Khaki, Junxian Guo, Jiaming Tang, Shang Yang, Yukang Chen, Konstantinos N. Plataniotis, Yao Lu, Song Han, and Zhijian Liu. 2025. Sparsevila: Decoupling visual sparsity for efficient vlm inference. In *ICCV*, pages 23784–23794.
- Moshe Kimhi, Omer Kerem, Eden Grad, Ehud Rivlin, and Chaim Baskin. 2025a. [Noisy annotations in semantic segmentation](#). *Preprint*, arXiv:2406.10891.
- Moshe Kimhi, Erez Koifman, Ehud Rivlin, Eli Schwartz, and Chaim Baskin. 2025b. [Waveclip: Wavelet tokenization for adaptive-resolution clip](#). *Preprint*, arXiv:2509.21153.
- Moshe Kimhi, Nimrod Shabtay, Raja Giryes, Chaim Baskin, and Eli Schwartz. 2026. [CARES: Context-aware resolution selector for VLMs](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2243–2256, San Diego, California, United States. Association for Computational Linguistics.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- Jewon Lee, Wooksu Shin, Seungmin Yang, Ki-Ung Song, DongUk Lim, Jaeyeon Kim, Tae-Ho Kim, and Bo-Kyeong Kim. 2025. Ergo: Efficient high-resolution visual understanding for vision-language models. *arXiv preprint arXiv:2509.21991*.
- Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Xin Zhao, and Ji-Rong Wen. 2023. Evaluating object hallucination in large vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 292–305. Association for Computational Linguistics.
- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. 2025. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*.
- Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xu-Cheng Yin, Cheng-Lin Liu, Lianwen Jin, and Xiang Bai. 2024. OCR-Bench: On the hidden mystery of OCR in large multi-modal models. *Science China Information Sciences*, 67(12).
- Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. 2022. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2263–2279. Association for Computational Linguistics.
- Minesh Mathew, Dimosthenis Karatzas, and C.V. Jawahar. 2021. DocVQA: A dataset for VQA on document images. In *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 2200–2209.
- Kwangrok Ryoo, Yeonsik Jo, Seungjun Lee, Mira Kim, Ahra Jo, Seung Hwan Kim, Seungryong Kim, and Soonyoung Lee. 2023. [Universal noise annotation: Unveiling the impact of noisy annotation on object detection](#). *Preprint*, arXiv:2312.13822.
- Haozhan Shen, Kangjia Zhao, Tiancheng Zhao, Ruochen Xu, Zilun Zhang, Mingwei Zhu, and Jianwei Yin. 2025. Zoomeye: Enhancing multimodal llms with human-like zooming capabilities through tree-based image exploration. In *Proceedings of the*

2025 Conference on Empirical Methods in Natural Language Processing.

Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. 2019. Towards vqa models that can read. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Alex Su, Haozhe Wang, Weiming Ren, Fangzhen Lin, and Wenhui Chen. 2025. Pixel reasoner: Incentivizing pixel-space reasoning with curiosity-driven reinforcement learning. *arXiv preprint arXiv:2505.15966*.

Pavan Kumar Anasosalu Vasu, Fartash Faghri, Chunliang Li, Cem Koc, Nate True, Albert Antony, Gokul Santhanam, James Gregory Gabriel, Peter Gräsch, Oncel Tuzel, and Hadi Pouransari. 2025. Fastvlm: Efficient vision encoding for vision language models. In *CVPR*.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Galouédec. 2020. [TRL: Transformers Reinforcement Learning](#).

Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, et al. 2025. Internvl3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. *arXiv preprint arXiv:2508.18265*.

Penghao Wu and Saining Xie. 2024. V*: Guided visual search as a core mechanism in multimodal llms. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13084–13094.

xAI. 2024. Realworldqa. <https://x.ai/blog/grok-1.5v>. Accessed: 2024.

Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, Jiaqi Wang, Feng Wu, and Dahua Lin. 2025. Conical visual concentration for efficient large vision-language models. In *CVPR*, pages 14593–14603.

Cheng Yang, Yang Sui, Jinqi Xiao, Lingyi Huang, Yu Gong, Chendi Li, Jing Nathan Yan, Yu Bai, Ponnuswamy Sadayappan, Xia Ben Hu, and Bo Yuan. 2025a. Topv: Compatible token pruning with inference time optimization for fast and low-memory multimodal vision language model.

Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. 2025b. Visionzip: Longer is better but not necessary in vision language models. In *CVPR*, pages 19792–19802.

Senqiao Yang, Junyi Li, Xin Lai, Bei Yu, Hengshuang Zhao, and Jiaya Jia. 2025c. Visionthink: Smart and efficient vision language model via reinforcement learning. In *NeurIPS*.

Weihao Ye, Qiong Wu, Wenhao Lin, and Yiyi Zhou. 2025. Fit and prune: Fast and training-free visual token pruning for multi-modal large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*.

Kaichen Zhang, Bo Li, Peiyuan Zhang, Fanyi Pu, Joshua Adrian Cahyono, Kairui Hu, Shuai Liu, Yuanhan Zhang, Jingkang Yang, Chunyuan Li, and Ziwei Liu. 2025a. LMMs-Eval: Reality check on the evaluation of large multimodal models. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 881–916. Association for Computational Linguistics.

Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. 2025b. Sparsevlm: Visual token sparsification for efficient vision-language model inference. In *ICML*.

Ziwei Zheng, Michael Yang, Jack Hong, Chenxiao Zhao, Guohai Xu, Le Yang, Chao Shen, and Xing Yu. 2026. Deepeyes: Incentivizing "thinking with images" via reinforcement learning. In *ICLR*.

Xin Zou, Di Lu, Yizhou Wang, Yibo Yan, Yuanhuiyi Lyu, Xu Zheng, Linfeng Zhang, and Xuming Hu. 2025. Don't just chase "highlighted tokens" in mllms: Revisiting visual holistic context retention. In *NeurIPS*.

This supplementary document provides additional details, analyses, and visual examples that complement the main paper. We organize the material as follows: Section [A](#) presents supplementary visual examples from our data curation pipeline, including both successful annotations and failure cases of the automatic process. In addition, we analyze alternative data curation strategies using ANLS-based filtering, along with grounding model robustness. Section [B](#) provides qualitative examples of AwaRes at inference time, illustrating both successful predictions and failure modes. Section [C](#) details our latency analysis, including hardware setup, wall-clock measurements comparing AwaRes and VisionThink, and a response length analysis explaining the observed efficiency gains. Section [D](#) offers an in-depth analysis of the cold-start (SFT) stage, examining the coupled-decision policy (CDP) behavior, sensitivity to the tool-turn weight w_t , and tool-call formatting reliability. Section [E](#) presents GRPO ablations covering the cold-start stage, tool-cost penalty, and correctness reward, as well as how GRPO learns beyond grounding model annotations. Section [F](#) provides comprehensive training hyperparameters and configuration details. Finally, Section [G](#) includes the complete

prompts used throughout our pipeline: the LLM-as-a-Judge prompt for data curation, the grounding model localization prompt for grounding, and the system prompt used during SFT, GRPO, and inference.

A Data Annotation

A.1 Data Curation Pipeline

In this section, we provide in Figure 6 supplementary visual examples from our data curation pipeline. Additionally, in Figure 7 we illustrate failure cases where the automatic process did not produce satisfactory results.

A.2 Agreement of resolution selection

The two panels compare resolution-selection labels from our default judge LLAMA-3.3-70B (Grattafiori et al., 2024) against DEEPSEEK-V3.2 (Liu et al., 2025) (left) and ANLS (right), with LR and HR marking low- and high-resolution labels. The two LLM judges show strong agreement (96.88% along the diagonal), whereas ANLS agrees far less (80.11%). This indicates that many responses are semantically equivalent despite differing in phrasing, which ANLS penalizes - strengthening our decision to use a judge rather than a simple text-comparison metric.

A.3 Training data statistics

Figure 9 shows the resolution distribution of our training data. DocVQA (Mathew et al., 2021) exhibits the highest resolutions, whereas the LLaVA-Multi (Jiang et al., 2024) subset contains the lowest. VisionThink-Smart (Yang et al., 2025c) and ChartQA (Masry et al., 2022) display a broad spread across resolutions, while TextVQA concentrations cluster around 1000 pixels along one axis. We cap all resolutions at 2000×2000 , as prior work (Kimhi et al., 2026) demonstrated that truncating DocVQA resolution has negligible impact on performance.

A.4 ANLS-Based Data Curation Analysis

We compare annotating the crops using LaaJ vs. using ANLS in Tab. 2. ANLS is a string-oriented similarity measure designed for OCR-style exactness, and as such it is ill-suited for supervising our resolution-sufficiency labels, specifically since we observe the VLM answers with crops act as augmented input and thus perturbed the answers a bit (even when they are semantically correct).

This effect is reflected empirically. Training with ANLS-based labels for cold-start yields substantially lower average accuracy than LaaJ-based labels (75.9 vs. 79.70), with pronounced drops on text- and detail-sensitive benchmarks such as ChartQA (70.5 vs. 77.0) and OCRBench (70.0 vs. 78.8). While ANLS labeling attains a lower RTR (0.38 vs. 0.49), the accompanying accuracy degradation indicates that it often suppresses crop requests even when additional resolution is required, rather than producing a better-calibrated policy. Overall, LaaJ provides a more reliable semantic correctness signal for annotation, which is critical for learning the CDP without over-penalizing benign surface-form variation.

A.5 Grounding model robustness

To assess the reliability of our grounding model, we compare the crop regions selected by two strong VLM grounding models, Qwen3-VL-235B-A22B (Bai et al., 2025a) and InternVL3.5-241B-A28B (Wang et al., 2025), on the subset of samples where high-resolution cropping is beneficial. As shown in Figure 10, the two grounding models exhibit strong agreement, with 88% of predictions concentrated along the diagonal, indicating that the selected crop regions are largely model-agnostic rather than artifacts of a single grounding model’s idiosyncrasies. The few off-diagonal cases cluster among spatially adjacent regions (e.g., bottom-right vs. top-right), suggesting that disagreements stem from boundary ambiguity rather than fundamentally divergent localization.

B Benchmark Qualitative Examples

Figures 11 - 16 present positive examples where AwaRes successfully identifies and crops the image region relevant to the question. Each conversation shows the question, the tool call selected by the model, the predicted answer (matching the ground truth), the low-resolution input, and the retrieved high-resolution crop. Across charts, documents, and natural images, the retrieved high-resolution crops consistently isolate the task-relevant objects or text - such as axis labels, table cells, brand logos, and foreground subjects, enabling the model to extract fine-grained details that are unresolvable in the low-resolution overview alone.

B.1 Failure cases

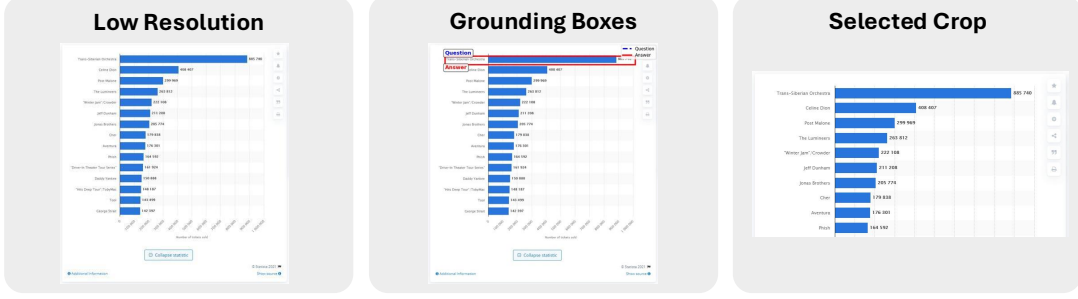
Figure 17 shows representative failure cases where the model’s either crop incorrect region (e.g. noisy

Prompt:

Which tour topped the list with over 558 thousand tickets sold?

Ground Truth Answer:

Trans-Siberian Orchestra



Prompt:

What is the brand name on the screen?

Ground Truth Answer:

spesial



Figure 6: Visual examples from our data curation pipeline. Each row shows the low-resolution input image, the grounding model-detected bounding boxes (question region in blue, answer region in red), and the selected crop used for training. Top: A Chart example where the grounding model correctly localizes the relevant bar and its label. Bottom: A natural image example requiring OCR of a brand banner.

annotations (Kimhi et al., 2025a; Ryoo et al., 2023)), or correct crop that lead to a wrong answer. Each conversation shows the question, the tool call selected by the model, the predicted answer (in red) and the ground truth answer from the data, the low-resolution input, and the retrieved high-resolution crop. These failures typically arise when the question-relevant detail occupies a small or ambiguous portion of the image, causing the crop to capture nearby but insufficient context.

C Latency Analysis

We analyze the latency by first describing the hardware and measurement setup, then reporting detailed wall-clock results, and finally comparing response lengths to explain the observed efficiency gains.

C.1 Hardware and Measurement Setup

To compare VisionThink (Yang et al., 2025c) and AwaRes, we evaluate both methods using their respective evaluation implementations in Imms-eval (Zhang et al., 2025a), with both models running in native HuggingFace configuration. We measure wall-clock time (WC) from the start of generation until the final answer is produced (encompassing both turns when applicable). All measurements done on Nvidia-H100-80GB GPU.

C.2 Detailed Latency Results

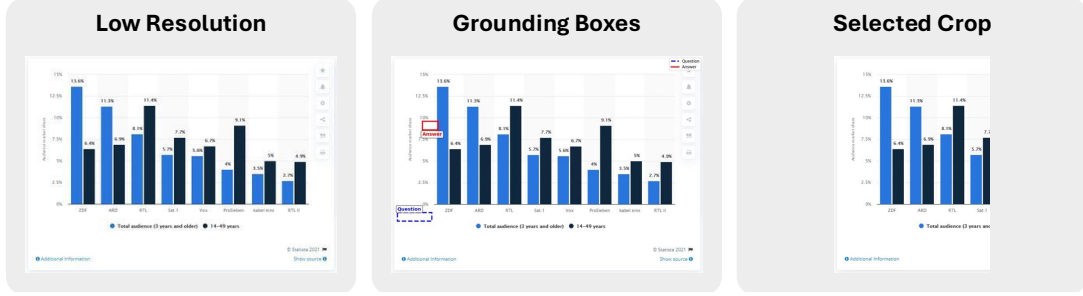
Table 3 presents the latency (WC) comparison between AwaRes and competing dynamic methods across six benchmarks. AwaRes achieves the lowest latency on every benchmark and the lowest average overall. On average, AwaRes reduces wall-clock time by $4.4\times$ compared to VisionThink (from

Prompt:

What was the market share of ZDF among 14 to 49 year olds?

Ground Truth Answer:

6.4



Prompt:

What country had the highest number of new HIV infections in 2015?

Ground Truth Answer:

South Africa

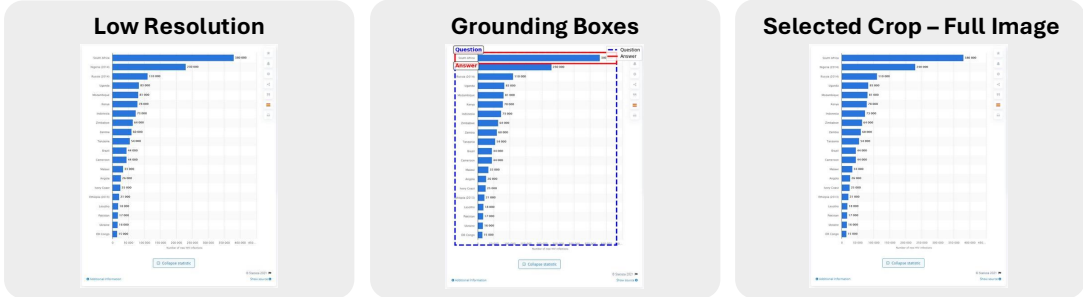


Figure 7: Failure cases from the automatic data curation pipeline. **Top:** The grounding model incorrectly localizes the answer region, missing the relevant bar (ZDF’s 14-49 age group value). However, the crop regions helps mitigate such localization errors-the selected crop still contains the correct answer. **Bottom:** The grounding model correctly identifies the question and answer regions but the bounding boxes span nearly the entire image, resulting in a full-image crop selection. While this produces correct supervision, it is inefficient as no resolution savings are achieved.

		LR		HR	
		DeepSeek-V3.2		ANLS	
LLaMA-3.3-70B	LR	78.01%	1.73%	69.04%	10.69%
	HR	1.39%	18.87%	9.19%	11.07%

Figure 8: **Agreement on resolution-selection labels.**

2.71s to 0.61s), 3.9 $\times$ compared to PixelReasoner (2.37s with low-resolution first turn), and over 9 $\times$ compared to DeepEyes (6.03s-7.22s), whose iterative visual reasoning incurs the largest overhead. Even against the strongest baseline, ERGO with a low-resolution first turn (1.20s average), AwaRes remains 2 $\times$ faster. The efficiency gains are most pronounced on memory- and reasoning-heavy benchmarks, where competing methods’ extended reasoning traces or repeated high-resolution passes incur substantial overhead.

C.3 Response Length Comparison

The latency differences stem primarily from response length. As shown in Table 4, VisionThink generates substantially longer reasoning traces, approximately 5.8 $\times$ to 28.8 $\times$ more characters than AwaRes. This verbosity directly translates to increased generation time, as autoregressive decoding scales linearly with output length.

Beyond raw efficiency, AwaRes exhibits significantly lower variance in response length across all benchmarks. This predictability has practical implications: estimated completion times become more reliable, enabling better resource allocation and user experience. In contrast, VisionThink’s high standard deviations—often exceeding the mean—make response length and latency highly unpredictable, complicating deployment in

Table 2: Comparing labeling strategies: LaaJ (LLaMA-3.3-70B) vs. ANLS. We report **Accuracy** $\uparrow$ and **RTR** $\downarrow$ across all benchmarks.

Label Strategy	ChartQA		DocVQA		OCRBench		POPE		RealWorld		V* Bench		Average	
	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$
ANLS	70.5	0.31	93.6	0.28	70.0	0.41	86.7	0.27	69.0	0.47	65.9	0.54	75.9	0.38
LLaMA-3.3-70B (LaaJ)	77.0	0.42	94.0	0.35	78.8	0.61	88.0	0.32	69.7	0.64	70.7	0.60	79.70	0.49

Table 3: **Latency comparison of dynamic methods.** Per-benchmark latency (seconds) for crop-inspired and adaptive methods. “LR” indicates the first turn uses a low-resolution image. AwaRes achieves the lowest average latency across all benchmarks. Best results per column in **bold**.

Model	ChartQA	DocVQA	OCRBench	POPE	RealWorld	V* Bench	Average
ERGO	1.82	3.18	1.16	1.09	2.42	2.60	2.05
ERGO (LR)	0.96	2.12	0.73	0.76	0.96	1.68	1.20
DeepEyes	6.15	7.48	5.13	8.19	7.03	9.33	7.22
DeepEyes (LR)	6.36	5.43	4.51	4.83	6.79	8.26	6.03
PixelReasoner	3.89	3.84	1.53	2.73	0.44	2.62	2.51
PixelReasoner (LR)	3.79	3.61	1.44	2.63	0.26	2.47	2.37
VisionThink	4.32	1.78	3.36	1.23	2.31	3.24	2.71
AwaRes	0.56	0.51	0.64	0.50	0.66	0.81	0.61

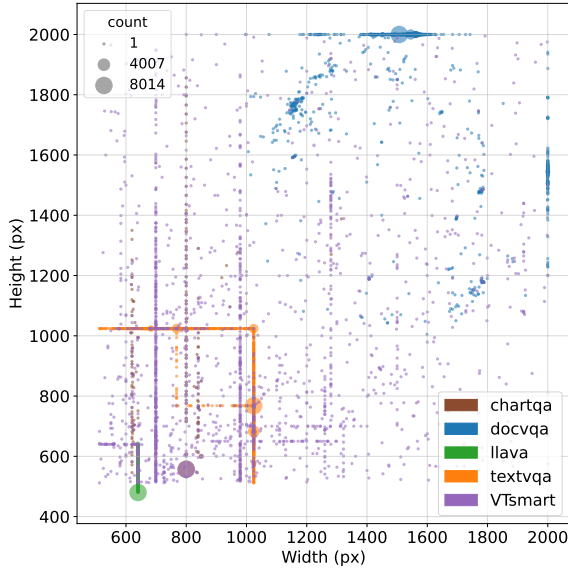


Figure 9: Resolution distribution of training datasets. Each point represents an image’s width and height in pixels.

latency-sensitive applications.

D Additional Cold-Start (SFT) Analysis

This supplementary section provides additional evidence that the cold-start stage learns a *coupled-decision policy (CDP)* whose first-turn action jointly determines *when* to request additional resolution ($C = \emptyset$ vs. $C \neq \emptyset$) and, when escalating, *where* to look via the chosen crop subset $C \subseteq \mathcal{C}$. Beyond final-task accuracy, we therefore report policy-centric diagnostics that quantify (i) the tendency to call the crop tool, (ii) failure to escalate

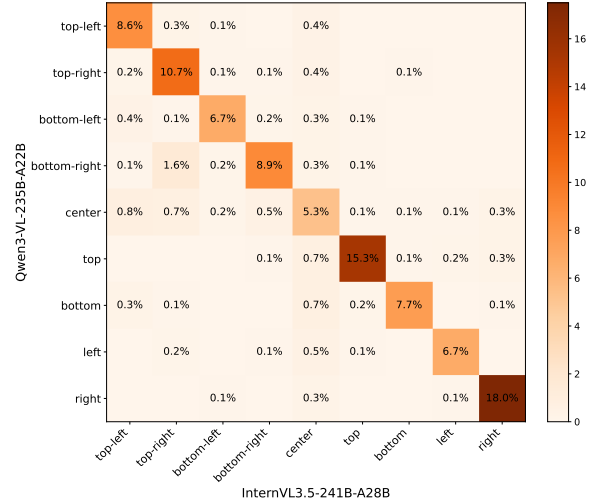


Figure 10: Crop confusion matrix between the two grounding model, conditioned on samples where high-resolution cropping helps. The diagonal concentration shows 88% agreement between the grounding models on the selected crop region.

Table 4: Response verbosity comparison between VisionThink and AwaRes across the benchmarks. We report the mean $\pm$ std number of characters in model responses. AwaRes produces substantially shorter responses than VisionThink ($5.8\times$ – $28.8\times$ reduction), reflecting its efficiency beyond visual token savings.

Benchmark	samples	VisionThink	AwaRes	Ratio
ChartQA	2500	89.46 $\pm$ 205.71	6.54 $\pm$ 6.16	13.7 $\times$
DocVQA (val)	5349	79.95 $\pm$ 242.21	13.74 $\pm$ 12.79	5.8 $\times$
OCRBench	1000	178.67 $\pm$ 540.12	16.23 $\pm$ 16.39	11.0 $\times$
POPE	9000	46.46 $\pm$ 155.52	2.84 $\pm$ 2.33	16.4 $\times$
RealWorldQA	765	160.67 $\pm$ 261.55	5.57 $\pm$ 5.86	28.8 $\times$
V*Bench	191	118.54 $\pm$ 232.17	5.29 $\pm$ 5.78	22.4 $\times$

when detail is required, and (iii) the amount of high-resolution evidence requested.

D.1 Metrics for CDP diagnostics

For each evaluated sample with resolution-sufficiency label $y \in \{\text{LR}, \text{HR}\}$, the model either takes a no-call action ($C = \emptyset$) or requests one or more crops ($C \neq \emptyset$). We evaluate the CDP as two coupled components:

Call decision (*when to crop*). We treat tool invocation as a binary classifier where the positive class is $y = \text{HR}$ and a prediction is positive iff $C \neq \emptyset$. We report:

- **Call Precision** $\uparrow$: $\mathbb{P}(y = \text{HR} \mid C \neq \emptyset)$.
- **Call Recall** $\uparrow$: $\mathbb{P}(C \neq \emptyset \mid y = \text{HR})$.
- **Call F1** $\uparrow$: $2 \text{Prec} \times \text{Rec} / (\text{Prec} + \text{Rec})$.
- **FPR (LR-call)** $\downarrow$: $\mathbb{P}(C \neq \emptyset \mid y = \text{LR})$.

Region decision (*where to look* | *call*). Conditioned on $C \neq \emptyset$, we measure overlap between the requested crop set and the grounding model target crops C^* used for SFT supervision. We report:

- **Exact match** ($\text{IoU} = 1$) $\uparrow$: the predicted region matches the grounding model target exactly.
- **Relaxed match** ($\text{IoU} \geq 0.25$) $\uparrow$: the predicted region overlaps a grounding model target by at least 0.25 IoU, accounting for the hierarchical structure of $\mathcal{C}$ (e.g., for a quadrant target, its two adjacent half-image regions are also considered acceptable; likewise, All and Center may be acceptable when they satisfy the IoU threshold).
- **Avg. area** $\downarrow$: $\mathbb{E}[s(C)]$, the average fraction of image area requested when calling.

In addition, we report **Accuracy** ($\uparrow$) and **RTR** ($\downarrow$) across all benchmarks, consistent with the main paper.

D.2 CDP behavior across cold-start variants

Table 5 decomposes cold-start behavior into the two components of the coupled-decision policy (CDP): the *call decision* (*when* to request crops) and the *region decision* (*where* to look, conditioned on calling).

Call decision (*when to crop*). Trajectory-level SFT substantially improves the calibration of the first-turn decision relative to baseline SFT. While baseline SFT achieves moderate precision (62.49) it exhibits very low recall (15.34) and a high false-positive rate on LR samples (79.69), indicating that tool invocation is both unreliable on HR cases and overly frequent on LR cases. Trajectory-level SFT increases recall to 41.02 and reduces FPR to 63.33, yielding a higher overall F1 score (24.63 $\rightarrow$ 44.56). Further upweighting the tool-call turn ($w_t=5$) strengthens this behavior: precision rises to 77.8, recall to 47.70, and FPR drops to 49.85, improving F1 to 59.14. Together, these results indicate that emphasizing the first-turn action stabilizes the fused CDP decision of whether to escalate.

Region decision (*where to look* | *call*). Trajectory-level SFT also improves alignment with grounding model’s supervision, increasing exact region match ($\text{IoU} = 1$) from 13.8 to 15.9 and relaxed overlap match ($\text{IoU} \geq 0.25$) from 32.6 to 48.85, while reducing the average requested area from 0.59 to 0.402. Upweighting the tool-call turn yields a large jump in localization quality ($\text{IoU} = 1$: 41.3; $\text{IoU} \geq 0.25$: 75.5), with a modest increase in requested area (0.402 $\rightarrow$ 0.463). This suggests that stronger supervision of the tool-call turn improves not only the decision to escalate but also the selection of informative regions once escalation occurs.

Overall, improved cold-start recipes shape both parts of the CDP: they improve the reliability of escalation (*when*) and the quality of evidence localization (*where*), while keeping the requested high-resolution area controlled. This motivates the subsequent GRPO stage, which further refines the same fused CDP under an explicit accuracy–efficiency objective.

Table 5: **Cold-start (SFT) policy diagnostics for the coupled-decision policy (CDP).** We evaluate the *call decision* as a binary classifier where the positive class is $y = \text{HR}$ and a prediction is positive iff the model calls the tool ($C \neq \emptyset$). We report precision, recall, and their harmonic mean (F1), as well as the false-positive rate on LR samples ($\text{FPR} = \mathbb{P}(C \neq \emptyset \mid y = \text{LR})$). For the *region decision* (conditioned on calling), we report overlap-based match rates to grounded targets using exact region match ($\text{IoU} = 1$) and relaxed overlap ($\text{IoU} \geq 0.25$), along with the average requested area. Higher is better ($\uparrow$) unless noted.

Model	CDP: call decision				CDP: region decision call		
	Call Prec. $\uparrow$	Call Rec. $\uparrow$	Call F1 $\uparrow$	FPR (LR-call) $\downarrow$	IoU=1 $\uparrow$	IoU ≥ 0.25 $\uparrow$	Avg. area $\downarrow$
SFT (baseline)	62.49	15.34	24.63	79.69	13.8	32.6	0.59
SFT + Traj.	48.75	41.02	44.56	63.33	15.9	48.85	0.402
SFT + Traj. + $w_t=5$	77.8	47.70	59.14	49.85	41.3	75.5	0.463

D.3 Cold-Start ablations

We analyze how cold-start SFT design choices shape both downstream performance and the induced coupled-decision policy (CDP). We isolate three factors—trajectory-level supervision, phased (tool-first) scheduling, and high-resolution upsampling—and study their interaction with the tool-turn weight w_t . Beyond accuracy and RTR, we examine how each setting affects when and how aggressively the policy escalates, as well as the reliability of the tool-call interface itself.

D.4 Cold-start sensitivity to data and parameters

Table 7 summarizes how common cold-start knobs shape both downstream performance and the induced coupled-decision policy (CDP). Trajectory-level SFT improves average accuracy over vanilla (77.90 vs. 75.15), but it also shifts the policy toward more frequent crop requests (call rate 22.55% $\rightarrow$ 25.21%) while substantially reducing the average requested area (0.59 $\rightarrow$ 0.402), indicating that the model learns to localize with smaller crops rather than relying on large regions. Increasing the tool-turn weight to $w_t=5$ further strengthens the first-turn CDP action, yielding the best accuracy (79.70) but also the highest RTR (0.49), consistent with a policy that escalates more often (call rate 29.02%) and requests slightly larger regions (Avg. area 0.463).

Beyond w_t , data and schedule choices can trade off calibration and efficiency. In particular, HR upsampling is expected to reduce missed escalations by exposing the policy to more detail-critical instances, but may increase tool usage if applied aggressively; phased (tool-first) schedules can stabilize tool invocation but sometimes change how

strongly the policy couples region selection to answer quality. Overall, the best cold-start configuration is the one that yields a reliable CDP (low misses and good region selection) while keeping call rate and requested area controlled, leaving GRPO to fine-tune efficiency rather than compensating for frequent cold-start failures.

D.5 Tool-call formatting reliability

Finally, Table 8 quantifies tool-call formatting reliability after cold-start by measuring whether a generated tool call can be parsed into a valid crop subset $C \subseteq \mathcal{C}$ without any post-processing. This isolates protocol learning from downstream accuracy: malformed outputs can prevent escalation even when the policy intends to call the tool, effectively breaking the CDP at the interface level.

Trajectory-level SFT substantially improves tool-call validity, reducing corruption from 10.17% to 1.43%. Moreover, the remaining failures are not merely cosmetic: a large fraction of corrupted calls lead to *incorrect* crop requests even after simple recovery (5.48% for baseline SFT vs. 0.94% for Traj.). Upweighting the tool-call turn ($w_t=5$) eliminates formatting corruption entirely (100% valid parse), supporting our design choice to explicitly emphasize the first-turn action during cold-start.

E GRPO ablations

We ablate the key design choices in our training pipeline: the role of the cold-start SFT stage, the structure of the tool-cost penalty, and the choice of correctness reward. Table 9 reports accuracy and retained token ratio (RTR) across six benchmarks.

Cold start vs. GRPO only. Training only the reference policy with SFT yields strong accuracy (79.70 average) but at a high compute cost (0.49 RTR), as the model has no incentive to crop economically. Conversely, running GRPO without a cold start drives RTR down sharply (0.31) but sacrifices accuracy (77.11), since the model never acquires a reliable cropping prior before reward optimization begins. AwaRes combines both stages, retaining the accuracy of SFT (80.30) while approaching the efficiency of GRPO-only (0.36 RTR), confirming that the two stages are complementary rather than redundant.

Tool-cost penalty. The area-based cost is what couples accuracy to efficiency. Removing the tool cost entirely raises RTR to 0.51 with no accuracy

Table 6: Cold-start ablations. We report **Accuracy** $\uparrow$ and **RTR** $\downarrow$ across all benchmarks.

Components				ChartQA		DocVQA		OCRBench		POPE		RealWorld		V* Bench		Average	
Traj.	Phased	Ups.	w_t	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$	Acc $\uparrow$	RTR $\downarrow$
x	x	x	1	72.9	0.35	93.1	0.42	73.7	0.32	85.8	0.29	64.1	0.33	61.3	0.42	75.15	0.36
v	x	x	1	69.8	0.38	93.8	0.32	76.9	0.54	87.8	0.30	70.5	0.55	68.6	0.48	77.90	0.43
v	x	v	1	76.9	0.30	93.2	0.27	72.9	0.41	84.7	0.25	69.0	0.43	66.0	0.55	77.12	0.37
v	v	x	1	70.9	0.36	93.4	0.40	76.1	0.32	86.6	0.30	68.8	0.34	64.4	0.41	76.70	0.36
v	v	v	1	70.8	0.33	92.7	0.37	72.7	0.31	85.7	0.28	66.8	0.33	63.9	0.43	75.43	0.33
v	x	x	1	69.8	0.38	93.8	0.32	76.9	0.54	87.8	0.30	70.5	0.55	68.6	0.48	77.90	0.43
v	x	x	5	77.0	0.42	94.0	0.35	78.8	0.61	88.0	0.32	69.7	0.64	70.7	0.60	79.70	0.49

Table 7: **Sensitivity of cold-start SFT.** We vary data/objective knobs and report both downstream performance (Avg. Accuracy and Avg. RTR across all benchmarks) and induced CDP behavior (call rate and average requested area, both measured over evaluation prompts). Call rate is $\mathbb{P}(C \neq \emptyset)$ and Avg. area is $\mathbb{E}[s(C)]$ (fraction of image area requested when calling).

Setting	HR upsample	w_t	Traj. SFT	Phased	Acc $\uparrow$	RTR $\downarrow$	Call rate	Avg. area $\downarrow$
Default SFT	x	1	x	x	75.15	0.36	22.55%	0.59
Traj. only	x	1	v	x	77.90	0.43	25.21%	0.402
Traj. + high w_t	x	5	v	x	79.70	0.49	29.02%	0.463
Traj. + HR upsample	v	1	v	x	77.12	0.37	23.0%	0.35
Phased (tool-first)	x	1	v	v	76.70	0.36	24.0%	0.44

Table 8: **Tool-call formatting reliability after cold-start.** Valid parse indicates that the tool output can be parsed into a crop subset $C \subseteq \mathcal{C}$ without post-processing. Corrupt outputs are split into *recoverable* cases (simple extraction of intended crop id(s) succeeds) and *incorrect* cases (post-processing yields a wrong crop).

Model	Valid parse (%) $\uparrow$	Corrupt outputs (%) $\downarrow$		
		Total	Recoverable	Incorrect
SFT (baseline)	89.83	10.17	4.69	5.48
SFT + Traj.	98.57	1.43	0.49	0.94
SFT + Traj. + $w_t=5$	100.00	0.00	0.00	0.00

benefit, as the model over-crops indiscriminately. Setting $\lambda = 0$ retains the misuse penalty but drops the area term, leaving RTR at 0.42—better than no cost, but still well above AwaRes’s 0.36. This shows the area-based term, not just the binary misuse signal, is responsible for the efficiency gains.

Correctness reward. We compare our text-similarity reward (sentence-transformer) against ANLS and LLM-as-a-judge (LaaJ). All three reach comparable accuracy (~ 80) and efficiency (~ 0.37 RTR), but text similarity offers the best trade-off between semantic matching and robustness to short-answer phrasing variation, without the latency and cost of an LLM judge in the reward loop.

E.1 GRPO learns beyond grounding annotations

The grounding model provides only noisy localization supervision for cold-start SFT: it does not provide final answers. The final policy is optimized by GRPO with answer-correctness and crop-cost

rewards, shifting from grounded model annotations towards better policy.

First, zero-shot grounding model tool use is insufficient. The strong grounding model drops from 88.9 average accuracy in native HR mode to 79.89 under the tool interface, with large drops on V^* ($90.6 \rightarrow 73.8$), ChartQA ($90.8 \rightarrow 69.7$), and OCRBench ($86.9 \rightarrow 77.8$). AwaRes, despite being 7B, reaches 80.30 average accuracy, and outperforms the tool-prompted grounding model (MOE, 235B-A22B) on ChartQA (80.6 vs. 69.7) and OCRBench (81.3 vs. 77.8). Thus, grounding model annotations alone do not solve the task. The learned crop distribution differs from the grounding model annotations, as shown in Figure 18

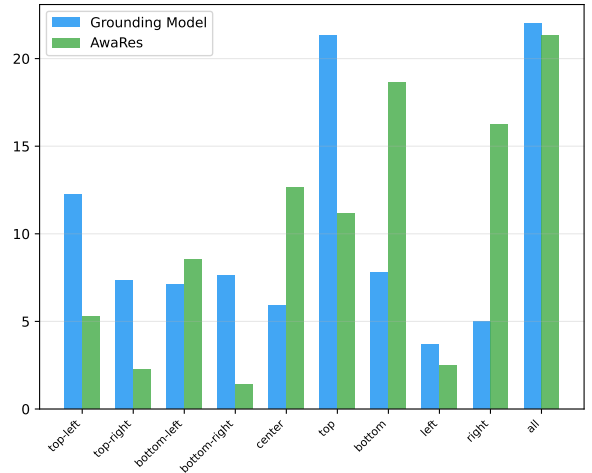


Figure 18: GRPO changes the crop policy relative to grounding model supervision instead of copying it’s crop choices.

F Training Details

We provide detailed hyperparameters for both training stages of AwaRes: Cold-Start SFT and Tool Optimization via GRPO. Table 10 conclude all the training parameters for both stages.

Table 9: **GRPO Ablations.** Top: comparing training the model only for the $\pi_{\theta_{ref}}$ (SFT) vs only GRPO (no cold start). Middle: effect of the tool cost penalty, $\lambda = 0$ removes the area-based cost while retaining the misuse penalty, and removing the tool cost entirely leads to further over-cropping. Bottom: comparison of correctness reward functions. Text similarity (sentence-transformer) balances semantic matching and robustness, outperforming both ANLS and LLM-as-a-judge on short-answer evaluation. RTR denotes the fraction of full-resolution compute used ($\downarrow$ is better).

Model	ChartQA		DocVQA		OCRBench		POPE		RealWorldQA		V* Bench		Average	
	Acc. $\uparrow$	RTR $\downarrow$	Acc. $\uparrow$	RTR $\downarrow$	Acc. $\uparrow$	RTR $\downarrow$	Acc. $\uparrow$	RTR $\downarrow$	Acc. $\uparrow$	RTR $\downarrow$	Acc. $\uparrow$	RTR $\downarrow$	Acc. $\uparrow$	RTR $\downarrow$
SFT only	77.0	0.42	94.0	0.35	78.8	0.61	88.0	0.32	69.7	0.64	70.7	0.60	79.70	0.49
GRPO only	76.48	0.25	93.98	0.25	72.20	0.31	85.91	0.25	67.06	0.32	67.02	0.54	77.11	0.31
w/o tool cost	80.60	0.44	94.60	0.37	81.50	0.64	87.80	0.34	69.40	0.65	71.20	0.62	80.85	0.51
w/o area cost ($\lambda = 0$)	81.28	0.38	94.42	0.37	80.70	0.40	85.93	0.37	69.28	0.41	71.20	0.58	80.47	0.42
Text-Sim $\rightarrow$ ANLS	79.84	0.29	94.35	0.25	80.00	0.41	85.78	0.23	69.67	0.43	70.16	0.57	79.96	0.37
Text-Sim $\rightarrow$ LaaJ	79.00	0.29	94.20	0.25	80.30	0.42	85.80	0.24	69.40	0.44	70.70	0.58	79.90	0.37
AwaRes	80.64	0.32	94.43	0.28	81.30	0.42	85.73	0.27	68.5	0.43	71.20	0.41	80.30	0.36

Table 10: Training hyperparameters for Cold-Start SFT and Tool Optimization (GRPO) stages.

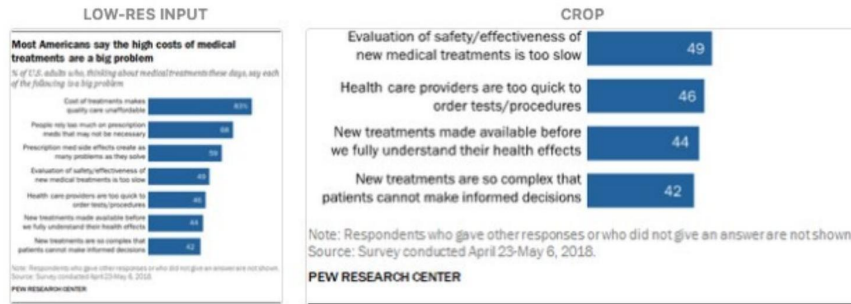
Parameter	Cold-Start (SFT)	Tool Opt. (GRPO)
<i>Model Configuration</i>		
Base model	Instruct HF model	Cold-Start model
LORA Rank	8	8
<i>Optimization</i>		
Optimizer	AdamW	AdamW
Learning rate	1e-4	5e-5
LR schedule	Cosine	Linear
Warmup ratio	0.05	0
Weight decay	0.001	0
Max gradient norm	-	1.0
<i>Batch Configuration</i>		
Per-device batch size	16	8
Effective batch size	128	64
Number of epochs	2	1
<i>Sequence Length</i>		
Max seq. / prompt length	8192	512
Max completion length	512	512
<i>Stage-Specific</i>		
Tool-turn weight w_t	5	-
Trajectory-level opt.	True	-
Number of generations G	-	8
Temperature τ	-	1.0
Top- p sampling	-	1.0
KL coefficient β	-	0.05
Crop cost weight $(1 - \alpha)$	-	0.25
Area weight $(1 - \alpha)$	-	0.01
Answer reward weight w_a	-	10.0

Question: What is the smallest value in the graph? Answer the question with a single word.

Tool: GET_CROPS [bottom]

Answer: 42

Ground Truth: 42

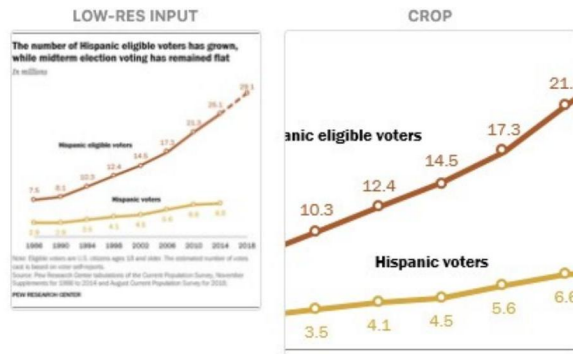


Question: What's the median value of red graph? Answer the question with a single word.

Tool: GET_CROPS [center]

Answer: 14.5

Ground Truth: 14.5



Question: What % of people say that everything possible should be done to make it easy to vote? Answer the question with a single word.

Tool: GET_CROPS [right]

Answer: 67

Ground Truth: 67

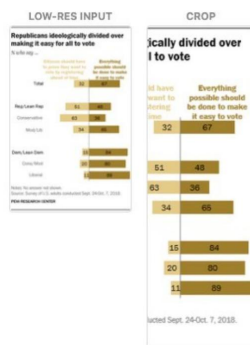


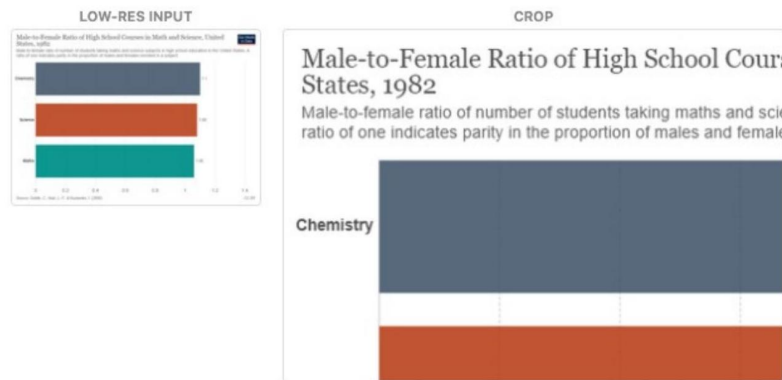
Figure 11: **Positive examples of AwaRes's adaptive cropping (1/6).** The model isolates the relevant bar segments and axis labels to identify the smallest chart value, zooms into data points along a trend line to compute a median, and focuses on labeled chart categories to read a specific percentage, extracting precise numerical values that are indiscernible in the low-resolution overview alone.

Question: Which subject has the highest male-to-female ratio of High School Courses? Answer the question with a single word.

Tool: GET_CROPS [top-left]

Answer: Chemistry

Ground Truth: Chemistry

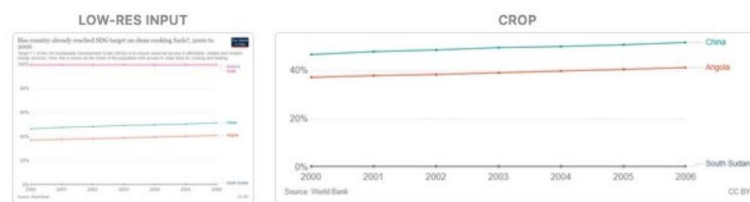


Question: Is the median of China in the graph greater than the median of the Angola graph? Answer the question with a single word.

Tool: GET_CROPS [bottom]

Answer: Yes

Ground Truth: Yes



Question: What race has the 2nd most people in Virginia? Answer the question with a single word.

Tool: GET_CROPS [top]

Answer: Black or African American alone

Ground Truth: Black or African American alone

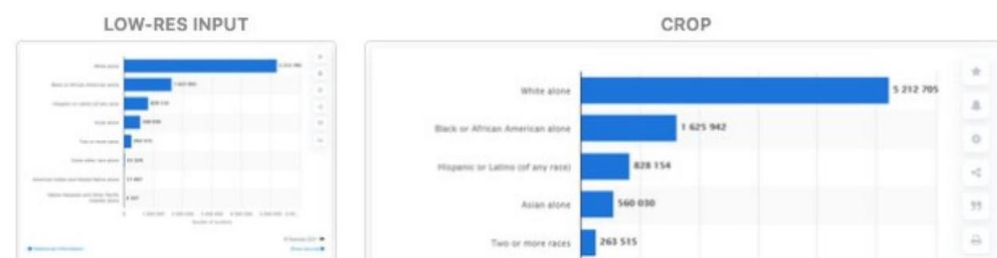


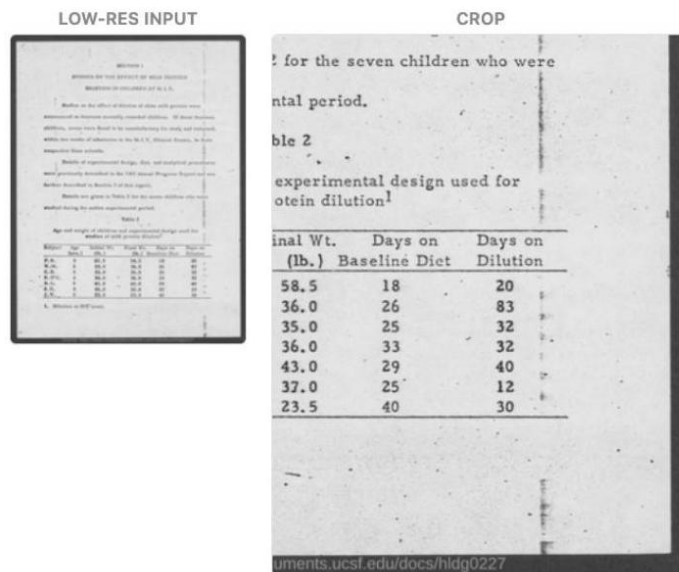
Figure 12: **Positive examples of AwaRes’s adaptive cropping (2/6).** The crops target the top bar and its label to identify the subject with the highest ratio, zoom into line-chart trends and country labels to compare medians across countries, and focus on the top entries of a horizontal bar chart to read demographic rankings. In each case, the high-resolution crop brings the answer-relevant region into sharp focus.

Question: What was the final wt. of subject S.D. ? Answer the question using a single word or phrase.

Tool: GET_CROPS [bottom-right]

Answer: 37.0

Ground Truth: ['37.0']



Question: what kind of wine is this? Reply with only one word or a short phrase.

Tool: GET_CROPS [bottom]

Answer: amarone

Ground Truth: ['amarone']



Figure 13: **Positive examples of AwaRes’s adaptive cropping (3/6).** The model crops into table cells of a scanned document to extract a specific weight value, and zooms into a wine bottle label to read the variety name. These examples demonstrate how targeted crops resolve fine-grained text on documents and labels that lack of OCR capabilities at low resolution.

Question: How many pedestrians are there within 20 meters from the camera? Please answer directly with a single word or number.
Tool: GET_CROPS [bottom]
Answer: 2
Ground Truth: 2



Question: How many cones are in this image? A. There are 2 cones in this image. B. There is 1 cone in this image. C. There are no cones in this image. Please answer directly with only the letter of the correct option and nothing else.
Tool: GET_CROPS [bottom]
Answer: A
Ground Truth: A



Question: In which direction is the dog in this scene traveling? A. From left to right B. From right to left Please answer directly with only the letter of the correct option and nothing else.
Tool: GET_CROPS [center]
Answer: A
Ground Truth: A

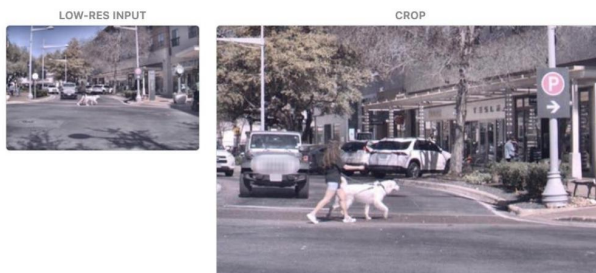


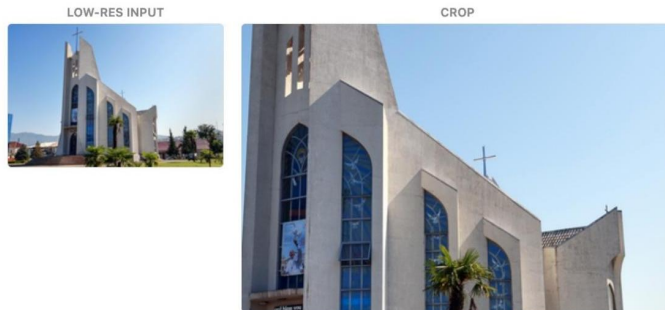
Figure 14: **Positive examples of AwaRes’s adaptive cropping (4/6).** The model correctly crops to count pedestrians in a rainy street scene, identify and count traffic cones in a parking lot, and determine the travel direction of a dog in an urban setting. The crops consistently center on the objects referenced by the question, enabling accurate spatial reasoning and counting from the high-resolution view of outdoor scenarios.

Question: What kind of animal is in the poster? A. dove B. horse C. dog D. cat Answer with the option's letter from the given choices directly. Answer with the option's letter from the given choices directly.

Tool: GET_CROPS [center]

Answer: A

Ground Truth: A

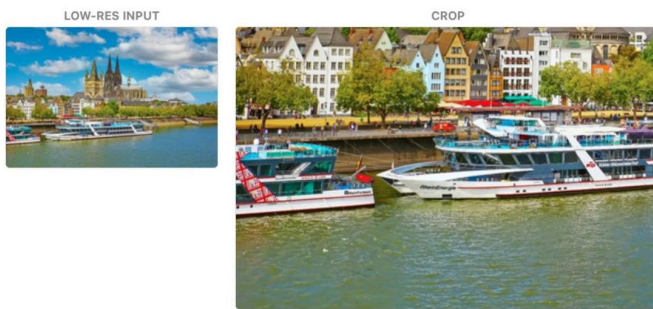


Question: What is the color of the van? A. red B. white C. silver D. black Answer with the option's letter from the given choices directly. Answer with the option's letter from the given choices directly.

Tool: GET_CROPS [bottom-left]

Answer: A

Ground Truth: A



Question: What is the color of the broom? A. black B. yellow C. gray D. red Answer with the option's letter from the given choices directly. Answer with the option's letter from the given choices directly.

Tool: GET_CROPS [right]

Answer: A

Ground Truth: A

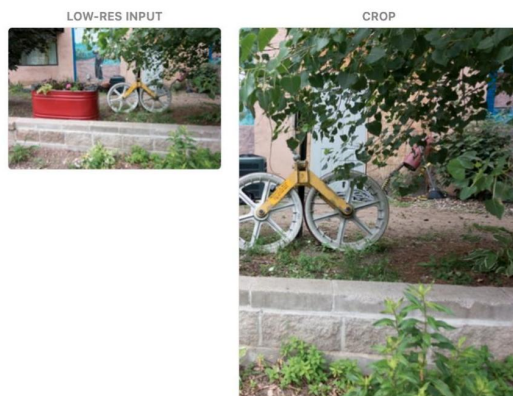


Figure 15: **Positive examples of AwaRes’s adaptive cropping (5/6).** The crops focuses on the church building top and detect the dove, zoom into the road next to the boats at a riverfront to resolve a van’s color, and focus on a broom among garden clutter to identify its color. These examples show how AwaRes localizes small or partially occluded objects that the question refers to, enabling fine-grained attribute recognition.

Question: What is the brand name of the 'Atta with multigrains' shown in the picture? Answer the question using a single word or phrase.

Tool: GET_CROPS [bottom]

Answer: Aashirvaad

Ground Truth: ['Aashirvaad']



Question: What is the color of the scarf? A. red B. green C. white D. yellow Answer with the option's letter from the given choices directly. Answer with the option's letter from the given choices directly.

Tool: GET_CROPS [center]

Answer: B

Ground Truth: B



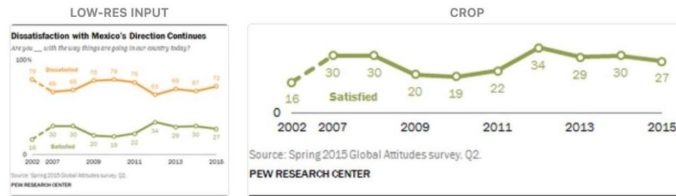
Figure 16: **Positive examples of AwaRes's adaptive cropping (6/6).** The model focuses on product packaging in a cluttered display to identify a brand name, and crops into the center of a snowy scene to zoom in on a person riding an ATV, resolving the green color of their scarf.

Question: What's the average of last three values in green graph (round to one decimal)? Answer the question with a single word.

Tool: GET_CROPS [bottom]

Answer: 30.3

Ground Truth: 28.6



Question: What is name of the sweetner which is used for Diet Slice? Answer the question using a single word or phrase.

Tool: GET_CROPS [top-Left]

Answer: Aspartame

Ground Truth: ['NutraSweet']

LOW-RES INPUT

CROP

INGREDIENT DATA

Pepsi-Cola Product Information

Ingredients are given per fluid ounce of bottled or canned beverage

Component ¹	Slice	Diet Slice	Mandarin
Calories	12.7	2.2	16.1
Water, gms.	27.4	29.0	25.6
Carbohydrates/Sugars, ² gms.	3.3	0.5	4.2
Phosphorus, mgs.	0	0	0
Caffeine, mgs. ³	0	0	0
Potassium, mgs. ⁴	8.3	8.3	10.0
Sodium, mgs. ⁴	0.9	0.9	1.8
Benzoic Acid, mgs.	8.7	8.7	9.5
Aspartame, mgs.	0	11.38	0

¹ These products contain negligible amounts of iron.

Only the finest inaredi

Question: What is the color of the telephone? A. green B. white C. orange D. blue Answer with the option's letter from the given choices directly. Answer with the option's letter from the given choices directly.

Tool: GET_CROPS [center]

Answer: B

Ground Truth: C

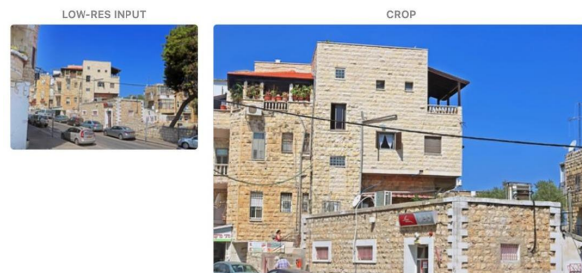


Figure 17: **Failure cases of AwaRes's adaptive cropping.** The crop either target the wrong region due to lack of sufficient context from the question, or produce a crop that targets a plausible region but fail to produce the correct answer. In the first example, the crop isolates the correct line, yet the model skips the value at 2013 and use 2012 instead, yielding the wrong average. In the second, the crop zooms into the Nutrition Facts label, instead of the ingredient list and reads "Aspartame" instead of the "NutraSweet". In the third, the crop focuses on the center, where the phone is located, but the granularity of the crop keep access of information around the object that makes the model misidentify its color.

G Prompts

G.1 LLM-as-a-Judge Prompt

LLM-as-Judge Prompt

You are tasked with evaluating which of two AI responses better answers the given question based on the provided ground truth.
Original prompt: {prompt}
Ground Truth Answer: {gt}
Response 1:
{lr_resp}
Response 2:
{hr_resp}

Compare both responses against the ground truth answer and respond with just one number, DO NOT include any other text:

- 0 - Both responses are equally good or equally bad, no significant difference
- 1 - Response 1 is better
- 2 - Response 2 is better

G.2 Grounding Prompt

Grounding model Localization Prompt

You are an expert visual assistant. Your task is to analyze an image and determine which regions are relevant for both understanding and answering a given question.

Instructions:

1. Analyze the question and identify what visual information is needed
2. Localize TWO types of regions:
 - a) QUESTION region: The area containing visual elements mentioned in or relevant to the question
 - b) ANSWER region: The area containing the answer to the question
3. For each region type, provide a bounding box in normalized coordinates [x_min, y_min, x_max, y_max]
 - Coordinates should be between 0 and 1000 (normalized to image dimensions)
 - x_min, y_min: top-left corner of the bounding box
 - x_max, y_max: bottom-right corner of the bounding box
4. Provide tight bounding boxes that closely encompass only the relevant text/visual elements
5. Output ONLY a Python dictionary with two keys: 'question' and 'answer' - NOTHING ELSE!!

CRITICAL: Your entire response must be ONLY a Python dictionary. Do not include any explanation, reasoning, or other text.

Output format:

```
{
  'question': [[x_min, y_min, x_max, y_max]],
  'answer': [[x_min, y_min, x_max, y_max]]
}
```

Notes:

- If the question and answer regions overlap significantly or are the same, you may provide the same coordinates for both
- If multiple regions are needed for either question or answer, include multiple bounding boxes in the list
- Keep bounding boxes as tight as possible around the relevant text or visual elements

Your response must be ONLY: {'question': [[x_min, y_min, x_max, y_max], ...], 'answer': [[x_min, y_min, x_max, y_max], ...]}

No other text, no explanation, just the dictionary.

Question: ...

G.3 SFT / GRPO / Inference Prompt

You are a vision-language model that analyzes images and answers questions about them. If the image resolution is too low for accurate analysis, respond with **GET_CROPS:** followed by a list of crop numbers in square brackets (e.g., GET_CROPS: ['3'] or GET_CROPS: ['0', '5']), where the available crop numbers and their corresponding areas are {'0': 'top-left', '1': 'top-right', '2': 'bottom-left', '3': 'bottom-right', '4': 'center', '5': 'top', '6': 'bottom', '7': 'left', '8': 'right', 'all': 'all'}, otherwise provide your answer.